

PROGRAM

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int np, nr;

    // Step 2 & 3: Input processes and resources
    printf("Enter the no: of processes\n");
    scanf("%d", &np);
    printf("Enter the no: of types of resources\n");
    scanf("%d", &nr);

    int exist[nr], max[np][nr], alloc[np][nr], need[np][nr];
    int possess[nr], avail[nr], completed[np], exec_seq[np];

    // Step 4 & 5: Input Existing vector and initialize Possessed to 0
    printf("Enter the total no: of instances of each type of resources\n");
    for (int i = 0; i < nr; i++) {
        printf("No: of instances of Resource%d: ", i);
        scanf("%d", &exist[i]);
        possess[i] = 0;
    }

    // Input Max Matrix
    printf("Enter the maximum no: of resources required by each process\n");
    for (int i = 0; i < np; i++) {
        for (int j = 0; j < nr; j++) {
            printf("Enter the maximum no: of R%d resources required by process%d\n", j, i);
            scanf("%d", &max[i][j]);
        }
    }

    // Step 6: Input Current Allocation, find Possessed Vector and Need Matrix
    printf("Enter the current allocation of resources required by each process\n");
    for (int i = 0; i < np; i++) {
        completed[i] = 0; // Clear completed status
        for (int j = 0; j < nr; j++) {
            printf("Enter the current allocation: of R%d resources to the process%d\n", j, i);
            scanf("%d", &alloc[i][j]);

            possess[j] += alloc[i][j]; // Calculate possessed resources
            need[i][j] = max[i][j] - alloc[i][j]; // Calculate need matrix
        }
    }

    // Step 7: Find Available Resource Vector
    for (int i = 0; i < nr; i++) {
        avail[i] = exist[i] - possess[i];
    }

    // Print Matrices as requested in manual outputs
    printf("\nMax Matrix\n");
    for (int i = 0; i < np; i++) {
        for (int j = 0; j < nr; j++) printf("%d\t", max[i][j]);
        printf("\n");
    }
}
```

```

}

printf("\nCurr alloc matrix\n");
for (int i = 0; i < np; i++) {
    for (int j = 0; j < nr; j++) printf("%d\t", alloc[i][j]);
    printf("\n");
}

printf("\nNeed matrix\n");
for (int i = 0; i < np; i++) {
    for (int j = 0; j < nr; j++) printf("%d\t", need[i][j]);
    printf("\n");
}

printf("\nExist Vector\n");
for (int i = 0; i < nr; i++) printf("%d\t", exist[i]);

printf("\n\nPossess Vector\n");
for (int i = 0; i < nr; i++) printf("%d\t", possess[i]);

printf("\n\nAvail Vector\n");
for (int i = 0; i < nr; i++) printf("%d\t", avail[i]);
printf("\n\n");

// Step 8: Banker's Algorithm Logic
int safe_flag = 0;
int seq_count = 0;

for (int k = 0; k < np; k++) {
    int found = 0;
    for (int i = 0; i < np; i++) {
        if (completed[i] == 0) {
            int can_complete = 1;
            // Step 8.a.i: Check unmet needs <= Available
            for (int j = 0; j < nr; j++) {
                if (need[i][j] > avail[j]) {
                    can_complete = 0;
                    break;
                }
            }
        }
    }

    // Step 8.a.ii & 8.a.iii: Mark completed, add to sequence, update Available
    if (can_complete == 1) {
        completed[i] = 1;
        exec_seq[seq_count++] = i;
        for (int j = 0; j < nr; j++) {
            avail[j] += alloc[i][j];
        }

        printf("Process%d can complete execution with available resources\n", i);
        printf("Avail Vector\n");
        for (int j = 0; j < nr; j++) {
            printf("%d\t", avail[j]);
        }
        printf("\n\n");

        found = 1;
    }
}

```

```

    }
}
}

// Step 8.b: If no process found, it's unsafe
if (found == 0) {
    safe_flag = 0;
    break;
}

// Step 8.c.i: If all processes finished, it's safe
if (seq_count == np) {
    safe_flag = 1;
    break;
}
}

// Step 9 & 10: Final Output
if (safe_flag == 1) {
    printf("The state is Safe and the sequence of process completion is ");
    for (int i = 0; i < np; i++) {
        printf("P%d ", exec_seq[i]);
    }
    printf("\n");
} else {
    printf("The state is Unsafe\n");
}

return 0;
}

```

OUTPUT

blackie@blackie-pc:~/GEC_PKD_IT/S4/OS_LAB/EXPERIMENT_7\$./a.out

Enter the no: of processes

5

Enter the no: of types of resources

4

Enter the total no: of instances of each type of resources

No: of instances of Resource0: 6

No: of instances of Resource1: 3

No: of instances of Resource2: 4

No: of instances of Resource3: 2

Enter the maximum no: of resources required by each process

Enter the maximum no: of R0 resources required by process0

4

Enter the maximum no: of R1 resources required by process0

1

Enter the maximum no: of R2 resources required by process0

1

Enter the maximum no: of R3 resources required by process0

1

Enter the maximum no: of R0 resources required by process1

0

Enter the maximum no: of R1 resources required by process1

2

Enter the maximum no: of R2 resources required by process1

1
Enter the maximum no: of R3 resources required by process1
2
Enter the maximum no: of R0 resources required by process2
4
Enter the maximum no: of R1 resources required by process2
2
Enter the maximum no: of R2 resources required by process2
1
Enter the maximum no: of R3 resources required by process2
0
Enter the maximum no: of R0 resources required by process3
1
Enter the maximum no: of R1 resources required by process3
1
Enter the maximum no: of R2 resources required by process3
1
Enter the maximum no: of R3 resources required by process3
1
Enter the maximum no: of R0 resources required by process4
2
Enter the maximum no: of R1 resources required by process4
1
Enter the maximum no: of R2 resources required by process4
1
Enter the maximum no: of R3 resources required by process4
0
Enter the current allocation of resources required by each process
Enter the current allocation: of R0 resources to the process0
3
Enter the current allocation: of R1 resources to the process0
0
Enter the current allocation: of R2 resources to the process0
1
Enter the current allocation: of R3 resources to the process0
1
Enter the current allocation: of R0 resources to the process1
0
Enter the current allocation: of R1 resources to the process1
1
Enter the current allocation: of R2 resources to the process1
0
Enter the current allocation: of R3 resources to the process1
0
Enter the current allocation: of R0 resources to the process2
1
Enter the current allocation: of R1 resources to the process2
1
Enter the current allocation: of R2 resources to the process2
1
Enter the current allocation: of R3 resources to the process2
0
Enter the current allocation: of R0 resources to the process3
1
Enter the current allocation: of R1 resources to the process3
1
Enter the current allocation: of R2 resources to the process3

0
 Enter the current allocation: of R3 resources to the process3
 1
 Enter the current allocation: of R0 resources to the process4
 0
 Enter the current allocation: of R1 resources to the process4
 0
 Enter the current allocation: of R2 resources to the process4
 0
 Enter the current allocation: of R3 resources to the process4
 0

Max Matrix

4	1	1	1
0	2	1	2
4	2	1	0
1	1	1	1
2	1	1	0

Curr alloc matrix

3	0	1	1
0	1	0	0
1	1	1	0
1	1	0	1
0	0	0	0

Need matrix

1	1	0	0
0	1	1	2
3	1	0	0
0	0	1	0
2	1	1	0

Exist Vector

6	3	4	2
---	---	---	---

Possess Vector

5	3	2	2
---	---	---	---

Avail Vector

1	0	2	0
---	---	---	---

Process3 can complete execution with available resources

Avail Vector

2	1	2	1
---	---	---	---

Process4 can complete execution with available resources

Avail Vector

2	1	2	1
---	---	---	---

Process0 can complete execution with available resources

Avail Vector

5	1	3	2
---	---	---	---

Process1 can complete execution with available resources

Avail Vector

5	2	3	2
---	---	---	---

Process2 can complete execution with available resources

Avail Vector

6 3 4 2

The state is Safe and the sequence of process completion is P3 P4 P0 P1 P2

blackie@blackie-pc:~/GEC_PKD_IT/S4/OS_LAB/EXPERIMENT_7\$